

Experiment-2-Beginner-2

Title: FPGA-Based Congestion-Aware Packet Router with Dynamic Arbitration

Objective:

The objective of this experiment is to design and verify a five-port Network-on-Chip (NoC) router in Verilog HDL for use as the basic building block of a two-dimensional mesh. The router receives 40-bit flits on its LOCAL, NORTH, SOUTH, EAST and WEST ports, stores them in per-port, per-virtual-channel input buffers, determines the output port using dimension-ordered X-Y routing, arbitrates for the crossbar using a round-robin switch allocator, and forwards the flits using wormhole switching.

The router also implements credit-based flow control, so that a flit is forwarded only when the downstream buffer has space. The design is validated through a self-checking ModelSim RTL testbench covering ten test scenarios, and a small top-level wrapper (router_top.v) is provided for demonstration on the Terasic DE10-Nano FPGA board using switches, a push button and LEDs.

Block Diagram:

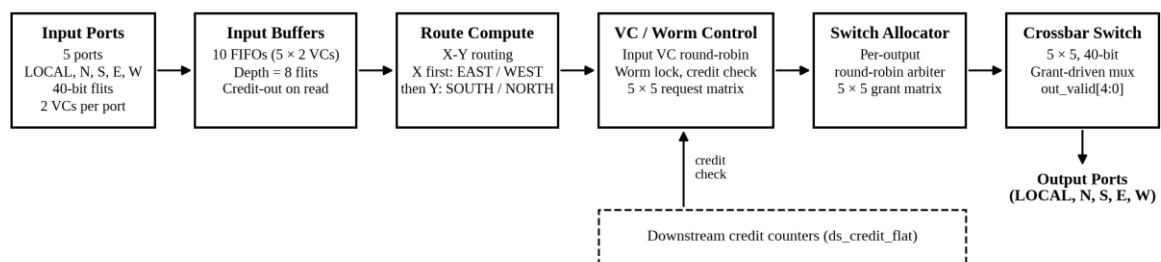


Fig. 1. Overall architecture of the 5-port, 2-VC wormhole NoC router.

A flit entering any of the five input ports is written into the FIFO selected by the one-hot virtual-channel (VC) signal of that port. The head flit at the front of each FIFO is passed to a route-compute unit, which compares the destination coordinates carried in the flit with the router coordinates and produces a one-hot output port. For every input port, the VC and wormhole control logic selects one VC in round-robin order, checks that the requested output VC is not locked by another packet and that the downstream credit counter is non-zero, and raises a request in a 5×5 request matrix. The switch allocator grants at most one input to each output port, and the crossbar switch copies the granted flit to the output port and asserts the corresponding out_valid bit. Each flit sent decrements the downstream credit counter of its output port and VC, and each credit returned through ds_credit_flat increments it.

RTL Design:

The router is described entirely in synthesizable Verilog HDL. The RTL consists of the `buffer.v`, `route_compute.v`, `switch_allocator.v`, `switch.v` and `router.v` modules. The testbench `router_tb.v` is used for functional verification, and `router_top.v` is a wrapper used for the board demonstration. The default parameters are a 40-bit flit, 4-bit coordinates, a buffer depth of 8 flits, and 2 virtual channels per port.

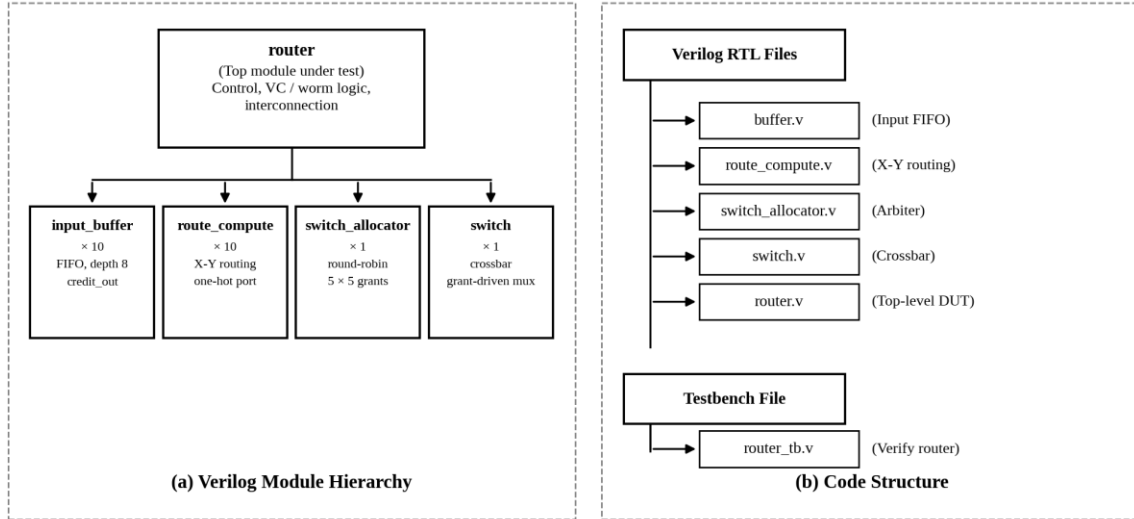


Fig. 2. Verilog module hierarchy and code structure of the NoC router.

The 40-bit flit format used by the router and the testbench is given below. The most significant bit marks the flit as valid, and the head and tail bits delimit a packet so that the router can lock and release a wormhole path.

Bits	Field	Description
[39]	Valid	Always 1 for a flit generated by the testbench
[38]	Head	Marks the first flit of a packet; triggers route computation
[37]	Tail	Marks the last flit of a packet; releases the wormhole path
[36:33]	Destination X	Destination column of the packet
[32:29]	Destination Y	Destination row of the packet
[28:0]	Payload	Data carried by the flit

1. buffer.v

`input_buffer` implements a synchronous FIFO with a parameterised depth (8 flits by default). It uses one extra pointer bit on both the write and read pointers to distinguish the full and empty conditions. A write is accepted only when the buffer is not full, and the read pointer advances only when the buffer is not empty. The flit at the head of the FIFO is always visible on `flit_out`, and `valid_head` is asserted when the buffer is non-empty and the head bit of that flit is set. The `credit_out` signal pulses whenever a flit is read, indicating that one buffer slot has been freed. The router instantiates ten of these buffers (five ports $\times$ two VCs).

2. route_compute.v

`route_compute` extracts the destination X and Y fields of a head flit and compares them with the router coordinates (`ROUTER_X`, `ROUTER_Y`) using signed subtraction. It implements deterministic X-Y routing: if the X offset is positive the output is EAST, if negative WEST; if X matches, a positive Y offset gives SOUTH and a negative Y offset gives NORTH; if both match, the packet is delivered to the LOCAL port. The result is a one-hot 5-bit code (`LOCAL` = 00001, `NORTH` = 00010, `SOUTH` =

00100, EAST = 01000, WEST = 10000). Resolving X before Y makes the routing deterministic and free of routing cycles on a mesh.

3. switch_allocator.v

switch_allocator receives a 5×5 request matrix and produces a 5×5 grant matrix. For every output port it maintains a round-robin pointer and grants the first requesting input starting from that pointer. After a grant, the pointer moves to the input following the winner, so that competing inputs are served fairly and no input is starved.

4. switch.v

switch is the 5×5 crossbar. For each output port, the granted input flit is copied to the corresponding slice of out_flit_flat and the matching bit of out_valid is set. When no grant is present for an output port, its flit is zero and its valid bit is low.

5. router.v

router is the top-level module of the design. It instantiates the ten input buffers, ten route-compute units, the switch allocator and the crossbar, and contains the control logic that connects them. For every input VC it keeps a worm_active flag and the stored output port of the packet currently being forwarded; for every output VC it keeps an out_vc_active lock. A head flit may be forwarded only if its output VC is free; once forwarded, the path is locked until the tail flit has passed, so that flits of different packets are never interleaved on the same output VC. A single-flit packet (head and tail set together) is forwarded without locking the path. Each output port and VC also has a downstream credit counter that is initialised to the buffer depth, decremented when a flit is sent and incremented by ds_credit_flat. A request is raised only if the counter is non-zero, which gives back-pressure when the downstream buffer is full. The VC identifiers are kept unchanged from hop to hop.

6. Testbench and Verification

router_tb is a self-checking testbench. The router under test is placed at coordinate (2,2) with a 10 ns clock and an active-low reset released after 40 ns. Tasks are used to build flits, inject packets on any port and VC, and monitor every output port. The monitor records each flit and VC that appears on the outputs and counts the credits returned on the input side. The expected packet contents, the number of flits at each output port, and the VC are compared automatically, and a [PASS] or [FAIL] message is printed for each check. The ten test scenarios are summarised below.

Test	Stimulus	Checked
TC1–TC4	3-flit packets from LOCAL to routers EAST (3,2), WEST (1,2), SOUTH (2,3) and NORTH (2,1)	Exact flit contents and VC at the expected output only
TC5	2-flit packet from LOCAL addressed to this router (2,2)	LOCAL loopback
TC6	Single-flit packet to EAST followed by a 2-flit packet to WEST	Single-flit worm releases the path
TC7	Two 3-flit packets interleaved on VC0 (to EAST) and VC1 (to WEST)	VC separation and routing
TC8	Two 4-flit packets from LOCAL and NORTH contending for EAST	Both complete without interleaving
TC9	32 back-to-back 4-flit packets from LOCAL to EAST	128 transfers, correct data and order

Test	Stimulus	Checked
TC10	11-flit packet to EAST with downstream credit held at zero, then credits returned	Output stops after 8 flits; buffered flits released by credits

At the end of the run, the testbench also verifies that the number of credits returned by the input buffers equals the number of flits consumed on each input port and VC (163 on LOCAL VC0, 3 on LOCAL VC1 and 4 on NORTH VC0).

Simulation Results:

The RTL was simulated in ModelSim (Intel FPGA Starter Edition 10.5b). The waveform below shows the testbench signals at the instant a flit injected on the LOCAL port (drive_valid = 00001, in_vc_flat = 0000000001, i.e. VC0) leaves the router on the EAST port: out_valid = 01000, with out_vc_flat = 0001000000 identifying VC0 of the EAST output. The destination fields of the flit decode to (3,2), one hop to the east of the router at (2,2), which is consistent with the X-Y routing rule. The downstream credit input ds_credit_flat is all ones in this window, so the output is not stalled.

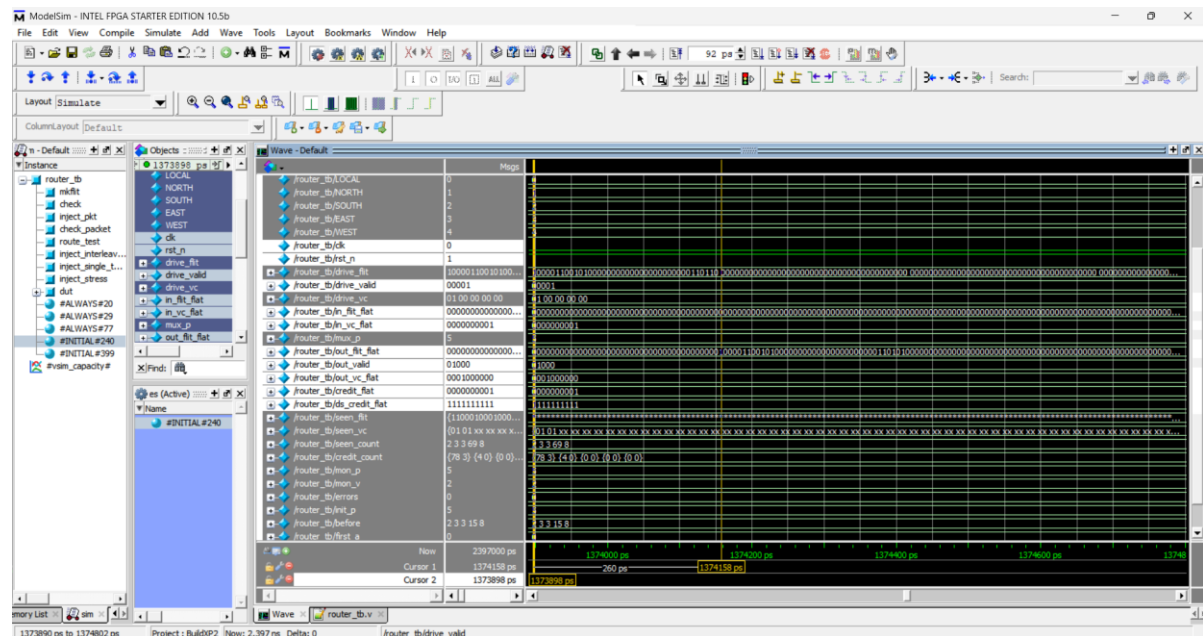


Fig. 3. ModelSim waveform of the router testbench showing a LOCAL-to-EAST transfer.

The transcript below shows the result of the complete run. All checks of TC1 to TC10 and the final credit check are reported as [PASS], the testbench prints ALL TESTS PASSED, and the simulation ends at 2397 ns.

```

Transcript
File Edit View Bookmarks Window Help

sim:/router_tb/out_flit_flat \
sim:/router_tb/out_valid \
sim:/router_tb/out_vc_flat \
sim:/router_tb/credit_flat \
sim:/router_tb/ds_credit_flat \
sim:/router_tb/seen_flit \
sim:/router_tb/seen_vc \
sim:/router_tb/seen_count \
sim:/router_tb/credit_count \
sim:/router_tb/mon_p \
sim:/router_tb/mon_v \
sim:/router_tb/errors \
sim:/router_tb/init_p \
sim:/router_tb/Before \
sim:/router_tb/first_a \
sim:/router_tb/packet \
sim:/router_tb/flit

VSM7> run -all
# ** Error: (vsim-7) Failed to open VCD file ".../sim/router_waves.vcd" in write mode.
# No such file or directory. (errno = ENOENT) : C:/Quartus_Projects/BuildX/router_tb.v(395)
# Time: 0 ps Iteration: 0 Instance: /router_tb
# [PASS] TC1 LOCAL to EAST exact packet
# [PASS] TC2 LOCAL to WEST exact packet
# [PASS] TC3 LOCAL to SOUTH exact packet
# [PASS] TC4 LOCAL to NORTH exact packet
# [PASS] TC5 LOCAL loopback exact packet
# [PASS] TC6 single-flit worm releases EAST
# [PASS] TC6 following packet routes WEST
# [PASS] TC6 no unexpected output
# [PASS] TC7 VCD reaches EAST once per flit
# [PASS] TC7 VC1 reaches WEST once per flit
# [PASS] TC7 no unexpected output
# [PASS] TC8 contending packets both complete
# [PASS] TC8 no unexpected output
# [PASS] TC9 32 back-to-back packets produce 128 transfers
# [PASS] TC9 no unexpected output
# [PASS] TC10 output stops when downstream credits reach zero
# [PASS] TC10 returned credits release buffered flits
# [PASS] TC10 no unexpected output
# [PASS] credit returns match consumed input flits
# ALL TESTS PASSED
# ** Note: $finish : C:/Quartus_Projects/BuildX/router_tb.v(397)
# Time: 2397 ns Iteration: 0 Instance: /router_tb
# Break in Module router_tb at C:/Quartus_Projects/BuildX/router_tb.v line 397
VSM8>

```

Fig. 4. ModelSim transcript showing all test cases passing.

Note: The transcript also contains a message that the VCD file `../sim/router_waves.vcd` could not be opened, because the folder `../sim` does not exist in the project directory. This only affects the optional VCD dump at time 0; it does not affect the checks, and the waveforms were viewed directly in the ModelSim Wave window.

Hardware Implementation:

For demonstration on the Terasic DE10-Nano board, `router_top.v` wraps a single router placed at coordinate (0,0). The board clock and reset drive the router, switches SW[3:2] select the destination X coordinate and SW[1:0] select the destination Y coordinate, and push button KEY1 (active-low, edge-detected) injects a single-flit packet through the LOCAL input on VC0. The downstream credit inputs are tied to zero because there is no external neighbour router, so the internal credit counters (initialised to 8) are the only limit on the number of flits that can leave each output.

The two LEDs display the output direction of the last forwarded packet: 00 for LOCAL, 01 for NORTH, 10 for SOUTH, and 11 for EAST or WEST. Because the destination coordinates from the switches are non-negative and the router is at (0,0), the packet can only be routed to LOCAL, EAST or SOUTH in this demonstration. The FPGA implementation and pin assignment are separate steps handled in Quartus; the results reported in this experiment are from RTL simulation only.



Applications:

The router developed in this experiment is the basic element of a mesh-based Network-on-Chip, which is used to connect processing elements, memory controllers and accelerators in multi-core processors and System-on-Chip designs. Virtual channels and credit-based flow control make it suitable for on-chip communication that requires fair sharing of links, back-pressure without data loss, and reduced head-of-line blocking.

The same router can be instantiated several times to build larger meshes for many-core systems, neural-network and other accelerator arrays, and FPGA-based multiprocessor prototypes, and it can be used as a platform to study routing, arbitration and flow-control techniques.

Conclusion:

The experiment successfully demonstrated a five-port, two-virtual-channel wormhole NoC router with X-Y routing, round-robin switch allocation and credit-based flow control. The RTL design was verified in ModelSim with a self-checking testbench: all ten test scenarios, including contention, back-to-back traffic and credit back-pressure, and the final credit-accounting check passed, and the simulation reported ALL TESTS PASSED. The work provided practical experience in NoC architecture, Verilog RTL design, flow control and simulation-based verification.